



**Federal University of Ceará**  
**Technology Center**  
**Computer Engineering Course**

## **Homework 2**

**Statistics for Engineers**

João Victor Carneiro de Oliveira, 569678

Fortaleza - CE

2025

# Contents

|          |                                                    |           |
|----------|----------------------------------------------------|-----------|
| <b>1</b> | <b>Questão 1</b>                                   | <b>2</b>  |
| 1.1      | Introdução Teórica . . . . .                       | 2         |
| 1.2      | Análise Gráfica . . . . .                          | 3         |
| 1.3      | Conclusão . . . . .                                | 4         |
| 1.4      | R Code . . . . .                                   | 4         |
| <b>2</b> | <b>2 Questão</b>                                   | <b>5</b>  |
| 2.1      | Introdução Teórica . . . . .                       | 6         |
| 2.2      | Análise gráfica . . . . .                          | 7         |
| 2.3      | Conclusão . . . . .                                | 7         |
| 2.4      | R Code . . . . .                                   | 8         |
| <b>3</b> | <b>3 Questão</b>                                   | <b>8</b>  |
| 3.1      | Theoretical Introduction . . . . .                 | 10        |
| 3.2      | Visual analysis . . . . .                          | 11        |
| 3.3      | Conclusion . . . . .                               | 12        |
| 3.4      | R Code . . . . .                                   | 12        |
| <b>4</b> | <b>Repository, references and additional files</b> | <b>13</b> |

## 1 Questão 1

Em um restaurante muito frequentado, aproximadamente 70% dos clientes pedem uma sobremesa após o prato principal. Seja  $X$  a variável aleatória que representa o número de clientes que pedem sobremesa em uma amostra aleatória de  $n = 50$  clientes.

- Determine a função de distribuição de  $X$ ;
- Construa os gráficos da função massa de probabilidade (PMF) e da função distribuição acumulada (CDF) de  $X$ ;
- Calcule o valor esperado, a variância e o desvio padrão de  $X$ ;
- Calcule a probabilidade de:
  - $P(X \geq 20)$
  - $P(30 < X < 43)$
  - $P(X = 31)$
- Suponha que o restaurante estoque sobremesas com base na demanda esperada. Como o uso da distribuição de  $X$  poderia ajudar a reduzir desperdício e evitar falta de produtos?
- Como mudanças em  $p$  (por exemplo, sobremesa se torna mais popular,  $p = 0.8$ ) ou em  $n$  (número de clientes) afetariam a forma e as probabilidades de  $X$ ?

### 1.1 Introdução Teórica

É notável que a variável aleatória  $X$  pode ser classificada como uma variável discreta, visto que o número de clientes pertence ao conjunto dos naturais. Como a decisão de cada cliente é, em uma análise simples, independente da decisão dos outros, com uma taxa aproximada de 70% para cada decisão individual, a função de distribuição se encaixa em uma distribuição binomial, na qual:  $X \sim \text{Bin}(50, 0.7)$ .

Para esse tipo de distribuição, o cálculo da Probability Mass Function segue abaixo:

- $P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}$

Já a esperança, a variância e o desvio padrão:

- Esperança =  $np = 50 \cdot 0.7 = 35$
- Variância =  $np(1 - p) = 50 \cdot 0.7 \cdot 0.3 = 10.5$
- Desvio Padrão =  $\sqrt{np(1 - p)} = \sqrt{10.5} = 3.24$

Os cálculos de probabilidades relativos aos respectivos intervalos são feitos a partir da análise dos valores individuais contidos em cada intervalo.

Para uma probabilidade tal que  $P(X \geq 20)$ , temos que calcular o complemento de  $P(X < 20)$ .  $P = 1 - P(19) + P(18) + \dots$

$$P = 0.999$$

Para um intervalo de probabilidade, usamos o fato que:  $P(30 < X < 43) = P(X < 43) - P(X < 30)$ :

$$P(X < 43) = 0.996$$

$$P(X < 30) = 0.085$$

$$P = 0.911$$

Por fim,  $P(X = 31)$  é encontrado pela própria fórmula da PMF:  $P(31) = \binom{50}{31} 0.7^{31} 0.3^{19} = 0.056$

## 1.2 Análise Gráfica

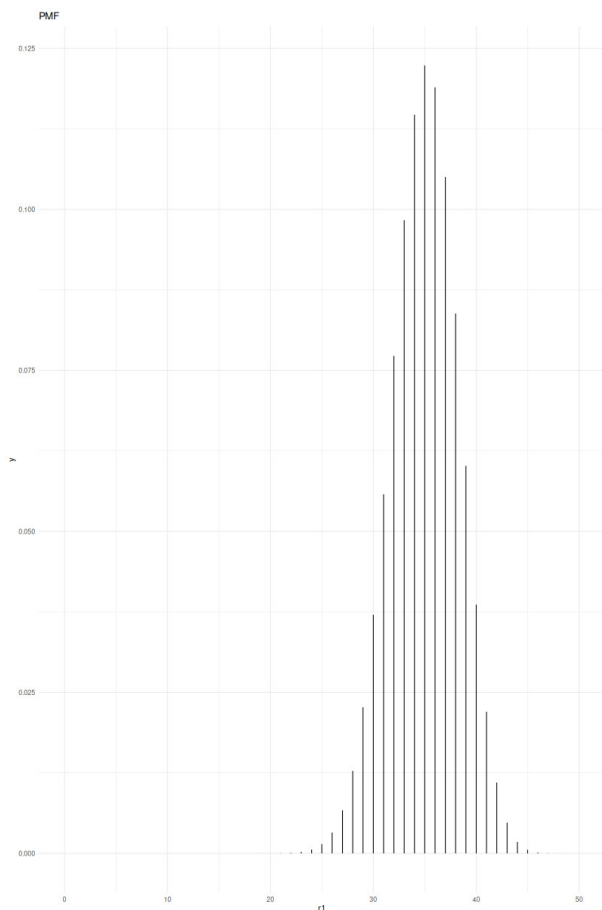


Figure 1: Função massa de probabilidade

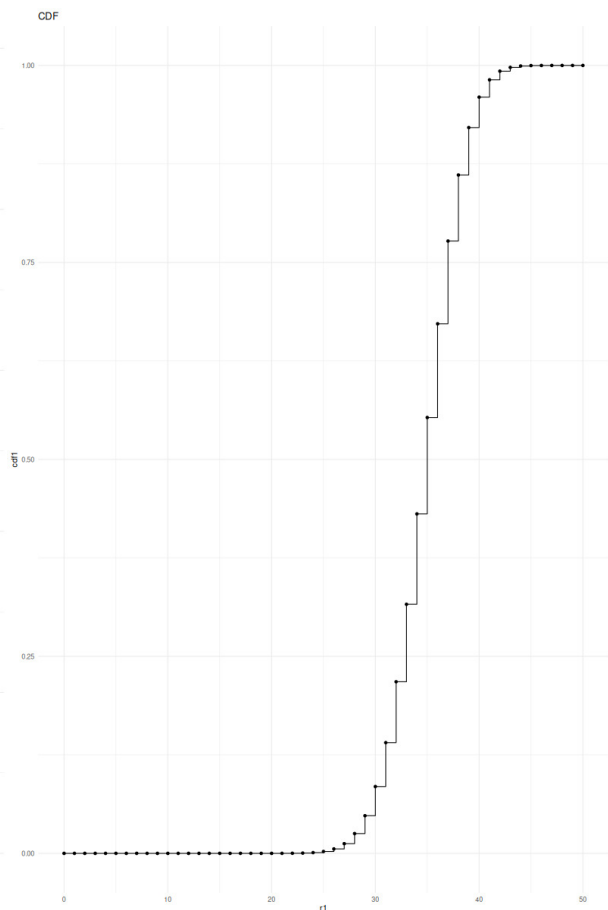


Figure 2: Função distribuição acumulada

### 1.3 Conclusão

Uma vez que a variável  $X$  é fielmente representada por uma função de distribuição binomial, sua forma tende a ser uma curva centrada no valor médio (Esperança), com distribuição simétrica em torno desse valor, disperso pela variância. Isto é, a forma da curva nos mostra a quantidade esperada de clientes que vão pedir sobremesa, sendo, para uma amostra de 50 clientes com  $p = 0.7$ , em torno de 35 clientes, com um desvio de aproximadamente 3 clientes para mais ou para menos. Ou seja, na maioria dos casos a quantidade clientes que vão pedir sobremesa nessas condições se concentra entre 32, 38 clientes.

Os cálculos de probabilidade encontrados também são coerentes com o formato da distribuição. Por exemplo, uma vez que os valores se concentram em torno de 35, é esperado que a probabilidade para  $X$  maior do que 20 seja quase a totalidade (100%).

Estes cálculos são importantes, pois assim, o restaurante sabe que um estoque muito acima de 38 pode acarretar em desperdício, enquanto um estoque abaixo de 32 pode não suprir toda a demanda. A análise da distribuição dos dados é crucial para o planejamento do estoque com antecedência, sabendo, assim, quanto preparar e estocar.

Uma mudança nos parâmetros provoca leves alterações na curva. Com uma sobremesa mais popular, por exemplo, o valor esperado crescerá (curva se desloca para direita) e o desvio padrão diminuirá (curva mais acentuada).

### 1.4 R Code

Os dados obtidos por meio dos códigos em R são precisos e coerentes com os dados em análises teóricas. É possível checar o código abaixo, ou em um repositório específico do Github ([homework2.R](#))

```

5  #parametros para a distribuição binomial da primeira questão
6  #sabemos que a análise se restringe à 50 clientes com probabilidade 70%
7
8  n1 <- 50
9  p1 <- 0.7
10 r1 <- 0:50 #range
11
12 pmf1 <- dbinom(r1, n1, p1)
13 cdf1 <- pbinom(r1, n1, p1)
14
15 #organizando os dados
16 d1 <- data.frame(r1, pmf1, cdf1)
17 head(d1)
18
19 #analise teórica
20 esperancal <- n1 * p1
21 variancial <- n1 * p1 * (1 - p1)
22 desvioPadrao1 <- sqrt(variancial)
23
24 esperancal
25 variancial
26 desvioPadrao1
27
28 #plot dos gráficos
29 #PMF
30 ggplot(d1, aes(x = r1, y = pmf1)) +
31   geom_segment(aes(xend = r1, y = 0, yend = pmf1)) +
32   labs(
33     title = "PMF"
34   ) +
35   theme_minimal()
36
37 #CDF
38 ggplot(d1, aes(x = r1, y = cdf1)) +
39   geom_step() +
40   geom_point() +
41   labs(
42     title = "CDF"
43   ) +
44   theme_minimal()

```

## 2 2 Questão

Um site realiza uma pesquisa *online* e oferece uma recompensa a um usuário selecionado aleatoriamente que responde a uma série de perguntas. Cada um dos 10 milhões de visitantes diários tem, independentemente, probabilidade  $p = 10^{-7}$  de ganhar a recompensa.

- Encontre uma aproximação simples e adequada para a função de massa de probabilidade (PMF) do número de vencedores em um dia,  $X$ . Justifique claramente se essa aproximação é apropriada

para os valores dados de  $n$  e  $p$ .

- Calcule o valor esperado,  $E[X]$ , e a variância  $Var(X)$ , usando tanto a distribuição exata quanto a aproximada. Comente sobre a semelhança entre os resultados.
- Suponha que você ganhe a recompensa, mas que possa haver outros vencedores. Seja  $W \text{ Pois}(1)$  o número de vencedores além de você. Se houver vários vencedores, o prêmio é sorteado aleatoriamente entre todos eles. Encontre a probabilidade de que você realmente receba o prêmio.
- Gere um grande número de simulações diárias para o número de vencedores. Crie uma comparação visual entre os resultados empíricos e a aproximação considerada no item ???. Descreva brevemente o que a visualização indica sobre a qualidade da aproximação.

## 2.1 Introdução Teórica

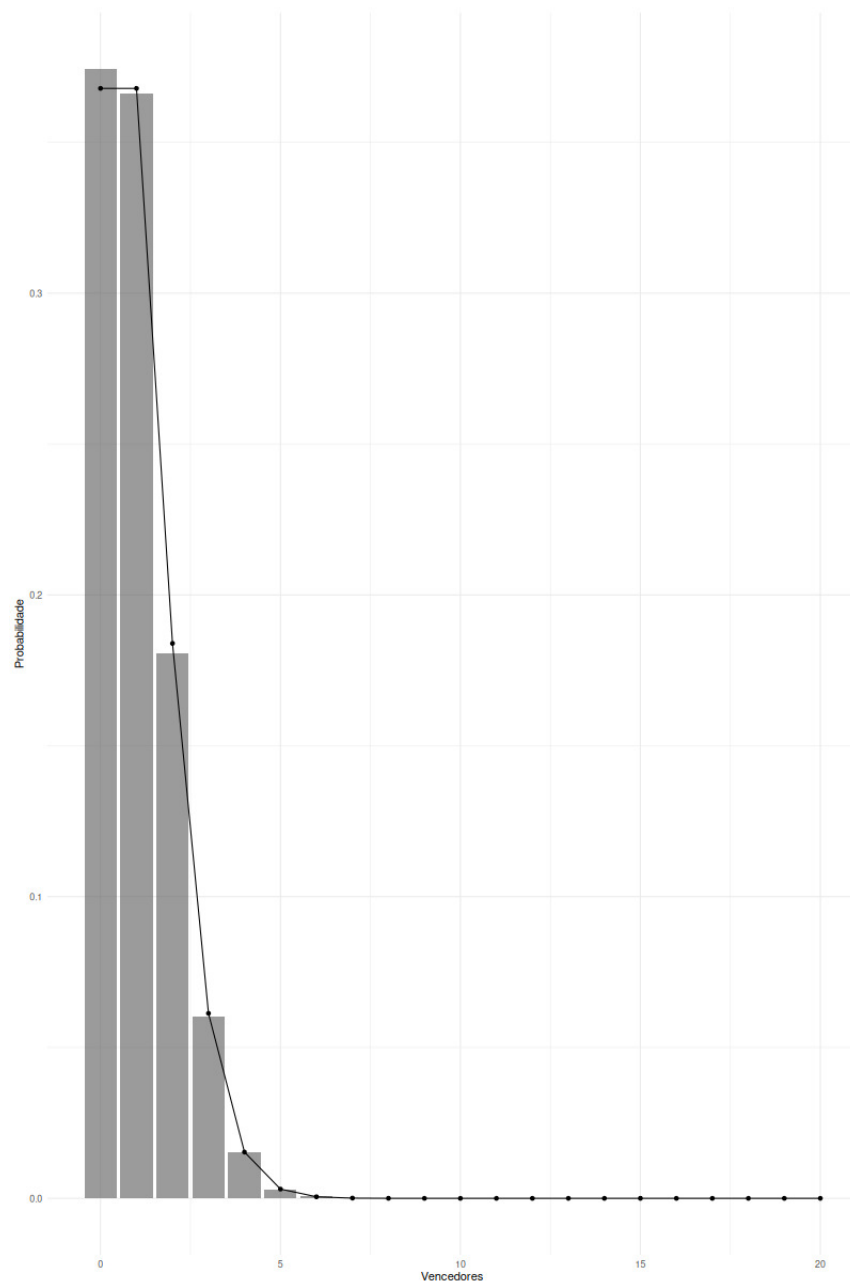
Assim como na questão anterior, a variável aleatória **discreta** deste problema tem uma distribuição binomial, posto que cada usuário tem a mesma probabilidade  $p$  de ganhar a recompensa, caracterizando tentativas independentes. Assim, poderíamos escrever  $X \text{ Bin}(10.000.000, 0.0000001)$ . Contudo, perceba que, como o valor de  $n$  é muito grande, e o valor de  $p$  muito pequeno, e que  $n$  é posto em visitantes diários (medida de tempo), a quantidade de usuários que ganham a recompensa também pode ser aproximada por uma distribuição de Poisson. Para a distribuição binomial:  $E = n \cdot p = 10^7 \cdot 10^{-7} = 1$ , logo, encontramos que em média 1 usuário ganha por dia. Isto equivale a uma distribuição  $X \text{ Poisson}(1)$ . Reiterando que essa aproximação faz-se válida pelos valores distantes de  $n$  e  $p$ , ou seja, uma chance muito pequena em meio a uma grande quantidade de usuários, assim como uma distribuição de Poisson.

- Pela distribuição binomial:  $E[X] = n \cdot p = 10^7 \cdot 10^{-7} = 1$  e  $Var(X) = npq = 10^7 \cdot 10^{-7} \cdot (1 - 10^{-7}) = 0.99999$ .
- Pela distribuição aproximada de Poisson:  $E[X] = Var(X) = \lambda = 1$ .

Os resultados das distribuições tem diferenças insignificantes para a análise dos dados, e ambas são coerentes.

Vamos supor que existam  $k$  outros ganhadores além do ganhador já certo dito  $V$ , indo de 0,  $10^7 - 1$ . Sabemos que a chance de  $V$  depende de  $k$  de modo que:  $P(V|k) = \frac{1}{k+1}$ . Por exemplo, para  $k=1$ ,  $P = 50\%$ . Dessa forma, devemos encontrar um valor médio de acordo com a probabilidade para cada valor de  $k$ , tal que:  $P = \sum_{k=0}^{10^7-1} P(V|k) \cdot P(k) = \sum_{k=0}^{10^7-1} (\frac{1}{k+1}) \cdot P(k)$ . Para uma distribuição de Poisson, com  $\lambda = 1$ :  $P(k) = \frac{e^{-1} 1^k}{k!}$ . Então:  $P(k) = \sum_{k=0}^{10^7-1} (\frac{1}{k+1}) \cdot \frac{e^{-1}}{k!} = e^{-1} \sum_{k=0}^{10^7-1} \frac{1}{(k+1)!}$ . Usando a aproximação da expansão da série de  $e$ , encontramos:  $P = 1 - \frac{1}{e} = 0.63$ . Os cálculos podem ser, posteriormente, revisados pela linguagem R.

## 2.2 Análise gráfica



## 2.3 Conclusão

A análise gráfica nos mostra que os valores são coerentes e próximos. (Em preto os valores encontrados para a aproximação de Poisson, em cinza para a distribuição binomial). Perceba que a partir de 5 vencedores diários a probabilidade é quase nula.



## 2.4 R Code

Os dados obtidos por meio dos códigos em R são precisos e coerentes com os dados em análises teóricas.

É possível checar o código abaixo, ou em um repositório específico do Github (homework2b.R)

```
4 #-----QUESTÃO 2
5 #parametros para a distribuição binomial da segunda questão
6 n <- 10000000
7 p <- 0.0000001
8 esperancaBinomial <- n * p
9 varianciaBinomial <- n * p * (1 - p)
10 #parametros para a distribuição de poisson
11 l <- esperancaBinomial
12 #valor médio considerando a distribuição de poisson
13 k <- 0:(10000000-1)
14 pois <- dpois(k, l) #valor da probabilidade para cada k poisson
15 poismedia <- sum((1 / (k + 1)) * pois)
16 poismedia #0.6321206
17 #simulação da binomial para vários dias
18 dias <- 10000
19 set.seed(222)
20 binomialmedia <- rbinom(d, n, p)
21 binomial <- table(binomialmedia) / dias
22 binomial_df <- data.frame(
23   k = as.numeric(names(binomial)),
24   prob = as.numeric(binomial)
25 )
26 #simulando no máximo 20 ganhadores para poupar
27 k <- 0:20
28 pois <- dpois(k, l) #valor da probabilidade para cada k poisson
29 poismedia <- sum((1 / (k + 1)) * pois)
30 pois_df <- data.frame(
31   k = k,
32   prob = pois
33 )
34 #comparação gráfica
35 ggplot() +
36   geom_col(data = emp_df,
37     aes(x = k, y = prob),
38     alpha = 0.6) +
39   geom_point(data = pois_df,
40     aes(x = k, y = prob)) +
41   geom_line(data = pois_df,
42     aes(x = k, y = prob)) +
43   labs(
44     x = "Vencedores",
45     y = "Probabilidade"
46   ) +
47   theme_minimal()
```

## 3 3 Questão

Você é responsável por monitorar a temperatura de uma *CPU multicore* em uma unidade de processamento embarcada. Sob carga normal, a temperatura da *CPU* apresenta flutuações devido a mudanças na carga de trabalho, nas condições ambientais e na eficiência do sistema de resfriamento. Testes mostram que a temperatura em regime estacionário da *CPU* segue uma distribuição normal com temperatura média  $\mu = 62^{\circ}\text{C}$  e desvio padrão  $\sigma = 3,5^{\circ}\text{C}$ . Sua tarefa é simular medições de temperatura da *CPU* e

analisar suas propriedades estatísticas.

- Crie uma função que gere valores com distribuição normal usando a transformação de *Box-Muller*, a partir de entradas aleatórias uniformes. Especificamente:
  - Gere duas variáveis aleatórias uniformes independentes:  $U_1, U_2 \sim Unif(0, 1)$ ;
  - Calcule dois valores normais padrão usando as fórmulas de *Box-Muller*:  $Z_1 = \cos(2\pi U_2)\sqrt{-2\ln(U_1)}$ ,  $Z_2 = \sin(2\pi U_2)\sqrt{-2\ln(U_1)}$ .  $Z_1, Z_2$  são variáveis aleatórias independentes com distribuição normal padrão. Concatene-as para formar um vetor  $Z$  de valores normais padrão;
  - Converta cada valor normal padrão para a distribuição de temperatura da CPU:  $T = 62 + 3.5Z$
- Use seu gerador de números aleatórios para gerar 1.000 medições de temperatura da *CPU*. Gere mais 1.000 valores de temperatura utilizando o gerador de números aleatórios normal embutido do R, com a mesma média e desvio-padrão.
- Para ambos os conjuntos de dados simulados, calcule:
  - Média amostral;
  - Desvio-padrão amostral;
  - Temperatura mínima e máxima amostrada;
  - Probabilidade empírica e teórica  $P(T < 68)$ ;
  - Probabilidade empírica e teórica  $P(60 < T < 65)$ ;
  - Probabilidade teórica  $P(T > 75)$ ;

Algum dos conjuntos de dados simulados (1.000 amostras) contém valores acima de  $75^\circ C$ ? Caso não, explique por que eventos raros requerem tamanhos de amostra grandes para serem observado.

- Visualize os resultados criando:
  - Um histograma das temperaturas simuladas da CPU (pode plotar os dois conjuntos de dados separadamente ou sobrepostos).
  - A função densidade de probabilidade (PDF) normal teórica (média  $62^\circ C$ , desvio padrão  $3,5^\circ C$ ) sobreposta ao histograma.
- Discuta seus resultados respondendo às seguintes perguntas: As distribuições empíricas da temperatura da *CPU* se assemelham à curva normal teórica? Quão próximas estão a média amostral e o desvio-padrão amostral dos valores esperados  $62^\circ C$  e  $3,5^\circ C$ ? Há diferenças perceptíveis entre o conjunto de dados gerado com seu *RNG* manual e o produzido pelo *RNG* embutido do R? Como essa simulação pode ajudar na avaliação de estratégias de resfriamento ou de escalonamento dinâmico de *clock*? Por que geradores de números aleatórios uniformes são a base dos sistemas de *RNG*?

### 3.1 Theoretical Introduction

A situação problema nos fornece um gráfico "em sino" contínuo que caracteriza tipicamente uma distribuição uniforme de uma variável contínua, nesse caso, temperatura. Contudo, as simulações computacionais requerem números aleatórios que são gerados em uma distribuição normal do tipo  $Unif(0, 1)$ , isto é, geram números aleatórios quaisquer entre 0 e 1, sendo o gráfico, portanto, uma linha reta, e não um "sino". Dessa forma, para simularmos a distribuição normal que representa os valores de temperatura da CPU utilizamos de um artifício chamado transformada de *Box-Muller*.

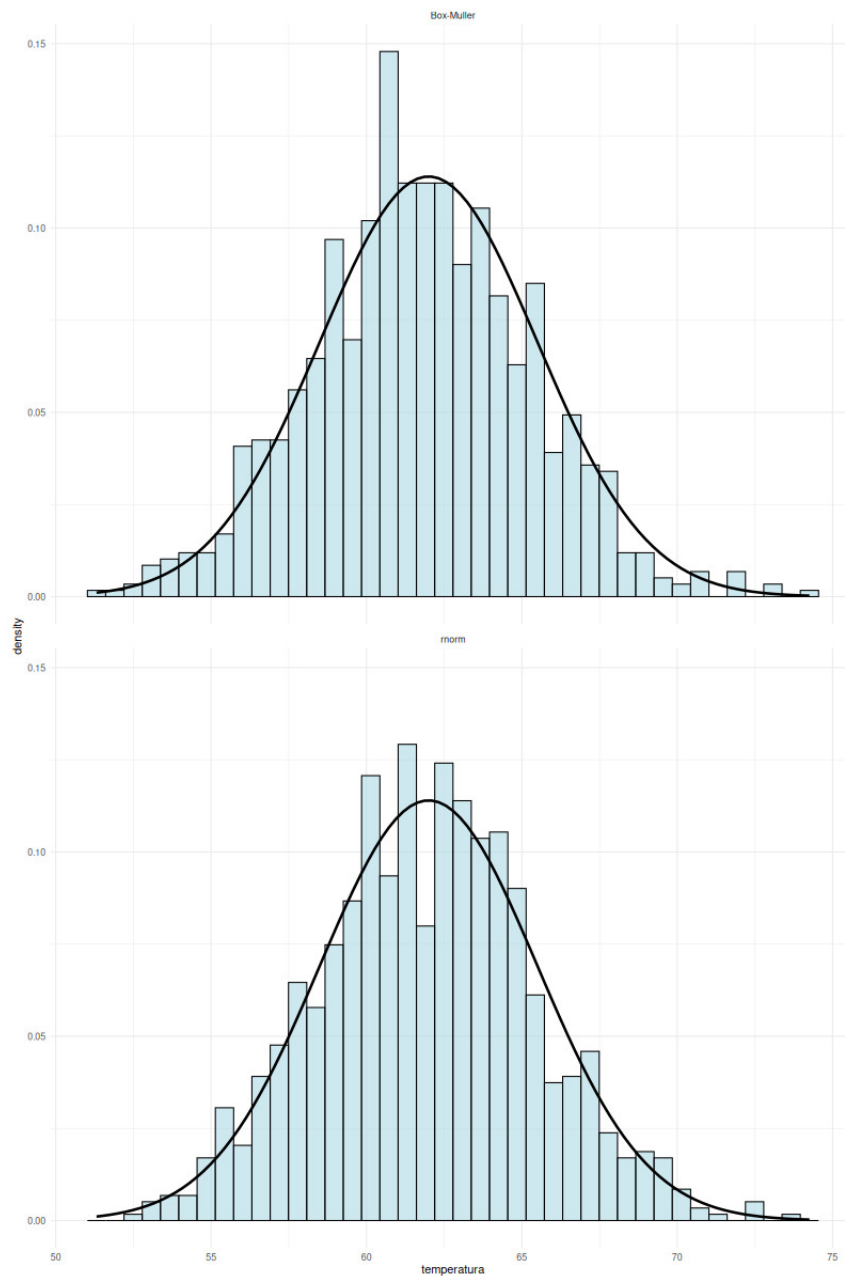
Primeiro geramos dois valores,  $U_1, U_2$  a partir de uma distribuição uniforme do tipo  $Unif(0, 1)$ . Depois, criamos um vetor com duas variáveis aleatórias de distribuição normal padrão,  $Z \sim N(0, 1)$ , a partir das formas de Box-Muller:  $Z_1 = \cos(2\pi U_2)\sqrt{-2\ln(U_1)}$ ,  $Z_2 = \sin(2\pi U_2)\sqrt{-2\ln(U_1)}$ . Por fim, ajustamos esse vetor para se adequar a distribuição de temperatura da CPU:  $T = 62 + 3.5Z$ . Os códigos e aplicações para essa transformada se encontram na seção *R Code*.

Usando, então, a distribuição gerada pela transformada, em conjunto com a distribuição da própria linguagem R, fazemos os cálculos devidos.

Para uma comparação teórica, sabendo que  $T \sim N(62, 3.5)$ :

- A média deve ser  $62^\circ C$ ;
- O desvio padrão deve ser  $3.5^\circ C$ ;
- O intervalo típico máximo e mínimo normalmente corresponde à:  $\mu \pm 3\sigma = 62 \pm 3(3, 5) = 51, 5^\circ C, 72, 5^\circ C$
- Aqui padronizamos a distribuição e usamos a função  $\phi$ , tal que:  $Z = \frac{T - \mu}{\sigma}$ . Para  $T < 68$  :  $Z = \frac{68 - 62}{3, 5} = 1, 71 \rightarrow P(Z < 1, 71) = 0, 956$ .
- Para  $T < 65$  :  $Z = \frac{65 - 62}{3, 5} = 0, 86 \rightarrow P(Z < 0, 86) = 0, 804$ . Já para  $T > 65$  :  $Z = \frac{60 - 62}{3, 5} = -0, 57 \rightarrow P(Z < -0, 57) = 0, 284$ . Tal que:  $P(60 < T < 65) = 0, 804 - 0, 284 = 0, 52$ .
- Por fim, para  $T < 75$  :  $Z = \frac{75 - 62}{3, 5} = 3, 71 \rightarrow P(Z < 75) = 0, 9999$ . Tal que:  $P(T > 75) = 1 - P(T < 75) = 1 - 0, 9999 = 0, 0001$ .

## 3.2 Visual analysis



- Para o conjunto da transformada:
  - A média foi: 61.74032
  - O desvio padrão: 3.50898
  - A temperatura mínima: 51.31467
  - A temperatura máxima: 74.25595
  - $P(T \leq 68)$ : 0.969

- $P(60 \leq T \leq 65)$ : 0.525
- $P(T \leq 75)$ : 0
- Para o conjunto gerado pelo R:
  - A média foi: 61.94536
  - O desvio padrão: 3.476375
  - A temperatura mínima: 52.28937
  - A temperatura máxima: 73.78673
  - $P(T \leq 68)$ : 0.957
  - $P(60 \leq T \leq 65)$ : 0.531
  - $P(T \leq 75)$ : 0
- Para nenhum houve qualquer valor acima de 75. Eventos tão raros quase nunca surgem em amostras pequenas simplesmente porque a chance de que apareçam é muito baixa. Análises de “cauda” exigem amostras grandes para serem observadas e avaliadas com confiabilidade. Os valores encontrados tanto nos cálculos teóricos, quanto na simulação pela transformada, quanto pela simulação nativa do R são extremamente próximos, garantindo uma confiabilidade triplicamente verificada.

### 3.3 Conclusion

As simulações mostram que as distribuições empíricas de temperatura ajustam-se bem à curva normal teórica: os histogramas têm formato de sino e a densidade teórica se sobrepõe de forma consistente. As médias e desvios-padrão amostrais ficam muito próximos dos valores esperados, como já mencionado, com pequenas variações naturais de amostragem. A distribuição `rnorm()` parece um pouco mais “suave”, mas ambos produzem essencialmente a mesma distribuição. A simulação é útil porque permite estimar, por exemplo, com que frequência a CPU ultrapassa limites seguros (como  $70\text{--}75^\circ\text{C}$ ), quanto tempo ela passa perto da temperatura nominal e qual fração do tempo ficaria em risco térmico. Com isso, podemos testar, sem queimar hardware, se vale a pena ativar o cooler mais cedo, reduzir o clock quando a temperatura ultrapassa certo limite ou dimensionar melhor dissipadores e ventilação. Usamos geradores uniformes como base porque é mais simples produzir números verdadeiramente “aleatórios” uniformes e, a partir deles, transformar matematicamente para qualquer outra distribuição, como a normal usada aqui.

### 3.4 R Code

Os dados obtidos por meio dos códigos em R são precisos e coerentes com os dados em análises teóricas. É possível checar o código abaixo, ou em um repositório específico do Github ([homework2c.R](#))

```

5 mu <- 62
6 sigma <- 3.5
7
8 transformada <- function() {
9   unif1 <- runif(1)
10  unif2 <- runif(1)
11
12  znormal1 <- sqrt(-2 * log(unif1)) * cos(2 * pi * unif2)
13  znormal2 <- sqrt(-2 * log(unif1)) * sin(2 * pi * unif2)
14
15  znormal <- c(znormal1, znormal2)
16
17  tnormal <- (znormal * 3.5) + 62
18  tnormal
19 }
20
21 #usando a transformada para gerar 1000 valores
22 tempfunction <- as.vector(replicate(500, transformada()))
23
24 #usando gerador de numeros aleatorios normal do R
25 set.seed(222)
26 tempr <- rnorm(1000, 62, 3.5)
27
28 #calculando o que se pede para a distribuição pela transformada
29 mean1 <- mean(tempfunction)
30 desvio1 <- sd(tempfunction)
31 tempmin1 <- min(tempfunction)
32 tempmax1 <- max(tempfunction)
33 p68e <- mean(tempfunction < 68)
34 p685e <- mean(tempfunction > 68 & tempfunction < 65)
35 p75e <- mean(tempfunction > 75)
36
37 mean1 #61.74032
38 desvio1 #3.50898
39 tempmin1 #51.31467
40 tempmax1 #74.25595
41 p68e #0.969
42 p685e #0.525
43 p75e #0
44
45 #calculando o que se pede para a distribuição normal
46 mean2 <- mean(tempr)
47 desvio2 <- sd(tempr)
48 tempmin2 <- min(tempr)
49 tempmax2 <- max(tempr)
50 p68t <- mean(tempr < 68)
51 p685t <- mean(tempr > 68 & tempr < 65)
52 p75t <- mean(tempr > 75)
53
54 mean2 #61.94536
55 desvio2 #3.476375
56 tempmin2 #52.28937
57 tempmax2 #73.78673
58 p68t #0.957
59 p685t #0.531
60 p75t #0
61
62 #criando os histogramas
63 dados <- rbind(
64   data.frame(temperatura = tempfunction, metodo = "Box-Muller"),
65   data.frame(temperatura = tempr, metodo = "rnorm")
66 )
67
68 ggplot(dados, aes(x = temperatura)) +
69   geom_histogram(aes(y = ..density..),
70     bins = 40,
71     fill = "#lightblue",
72     color = "black",
73     alpha = 0.6) +
74   stat_function(
75     fun = dnorm,
76     args = list(mean = mu, sd = sigma),
77     linewidth = 1.2
78   ) +
79   facet_wrap(~ metodo, ncol = 1) +
80   theme_minimal()

```

## 4 Repository, references and additional files

Os códigos estão disponíveis em um repositório do Github.

As seguintes páginas da *Internet* foram consultadas durante a redação desse relatório:

- Overleaf
- Overleaf - Learn
- R-Graph-Gallery
- Stackoverflow
- Wikipedia: Box-Muller transform
- R-Bloggers
- Scaler: rnorm()

O ChatGPT foi consultado com o seguinte prompt:

*O que usar em ggplot2 além de geom\_col pra gerar histogramas em R[sic]*

O slides disponibilizados no *SIGAA* também foram copiosamente consultados, agradeço a disponibilidade.